

Experiment No.	7
Experiment Title	Dimensionality Reduction and Model Evaluation (With and Without PCA) ¹
Student Name	Sugith Marini B
Register Number	3122247001065
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	31.08.2026

1 Objective

To study the effect of Principal Component Analysis (PCA) on the performance of 10 different machine learning classifiers on the Wisconsin Diagnostic Breast Cancer (WDBC) dataset. Every model is trained and evaluated in two settings – **No-PCA** (original 30 features) and **With-PCA** (reduced feature space retaining about 95% variance) – with hyperparameter tuning and 5-fold cross-validation used in both settings, so the two settings can be compared fairly.

2 Problem Statement

Given 30 numeric features computed from a digitized image of a breast mass fine needle aspirate (FNA), the task is to classify each sample as **Malignant (M)** or **Benign (B)**. This is a binary classification problem. In addition to building good classifiers, the experiment asks whether reducing the 30 features to a smaller set of principal components (using PCA) helps, hurts, or makes no real difference to classifier performance and stability, for 10 different types of classifiers.

3 Dataset Description

Dataset Name	Wisconsin Diagnostic Breast Cancer (WDBC)
Dataset Source	UCI Machine Learning Repository (<code>wdbc.data</code> , <code>wdbc.names</code>)
Number of Samples	569
Number of Features	30 numeric features (ID column dropped, not used as a feature)
Number of Classes	2 (Benign = 357 samples, 62.7%; Malignant = 212 samples, 37.3%)
Missing Values	0 (verified on the full dataset)
Duplicate Rows	0 (verified on the full dataset)
Train-Test Split	80:20 stratified split (Training = 455 samples, Test = 114 samples)

4 Exploratory Data Analysis

The only EDA step required for this experiment is checking the class balance, since the 30 features themselves are used directly (after scaling) for modelling.

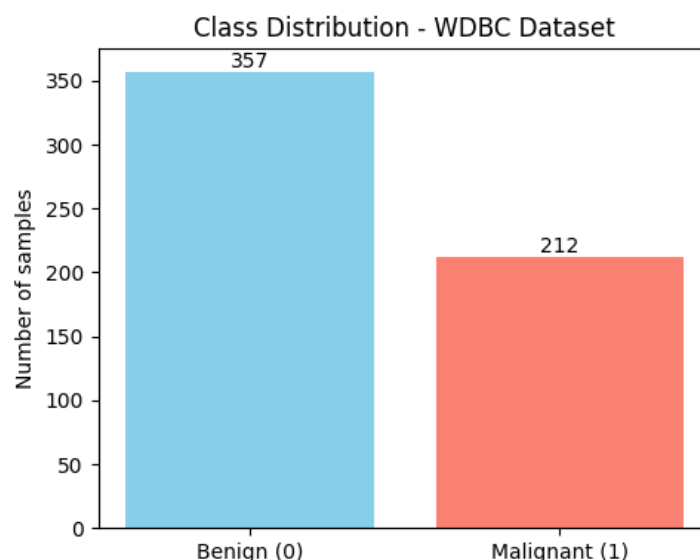


Figure 1: Class distribution of the WDBC dataset.

Inference: The dataset has 357 Benign and 212 Malignant samples (62.7% vs 37.3%). The classes are imbalanced but not severely so, which is why a stratified train-test split and stratified 5-fold cross-validation are used throughout this experiment, so that both classes are represented proportionally in every fold and in the test set.

5 Data Preprocessing

- **Missing values:** None found in the dataset, so no imputation was required.
- **ID column:** Dropped, since it is a sample identifier and not a predictive feature.
- **Encoding:** The diagnosis column was label-encoded, with Benign (B) = 0 and Malignant (M) = 1.
- **Feature scaling:** All 30 features were standardized using `StandardScaler` (zero mean, unit variance), fitted only on the training data to avoid data leakage.
- **Train-test split:** An 80:20 stratified split was used, giving 455 training samples and 114 test samples, with a fixed `random_state` for reproducibility.
- **Leakage prevention:** `StandardScaler` and `PCA` were both placed inside `scikit-learn` `Pipeline` objects, so they are fit only on the training portion of every cross-validation fold and never see validation-fold or test data during fitting.

6 Dimensionality Reduction Using PCA

PCA was applied with a variance target of approximately 95%, rather than an arbitrarily chosen fixed number of components. Fitting PCA (after scaling) on the training data shows that **10 out of the original 30 features** are needed to retain at least 95% of the total variance (95.21% exactly).

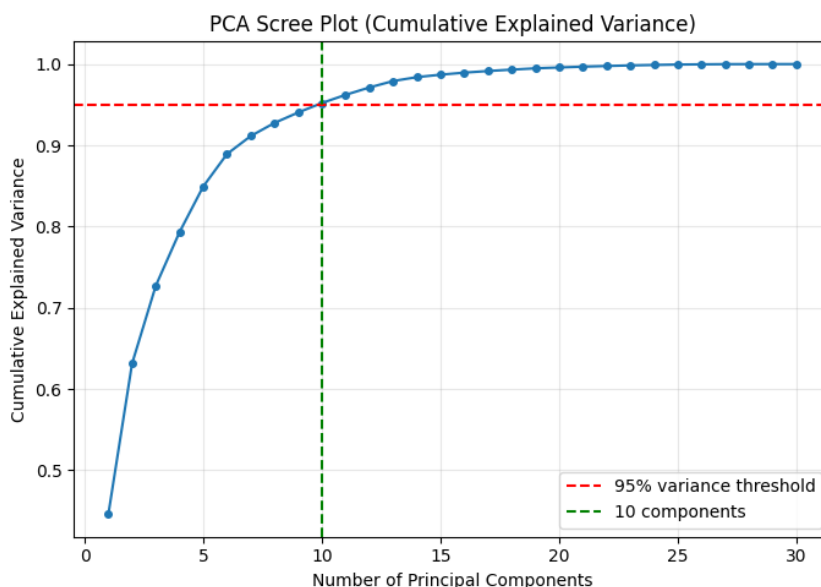


Figure 2: PCA scree plot: cumulative explained variance vs number of components.

Inference: The curve rises steeply for the first few components and flattens after about 10 components, showing that many of the 30 original features are correlated (several groups of features describe related nuclear-shape properties). PCA compresses this redundant information into a smaller set of uncorrelated components.

For the actual model training below, PCA is not applied using this single fixed component count. Instead, every “With-PCA” pipeline uses `PCA(n_components=0.95)` *inside* the pipeline, so PCA is refit independently on each training fold during cross-validation (each fold choosing its own number of components to reach 95% variance, which was consistently 10 in this experiment) rather than reusing one count computed in advance from the full training set. This avoids any information from outside a fold influencing that fold’s PCA transform.

Table 2: PCA Summary

Setting	Components / Variance Target	Explained Variance
No-PCA	All 30 original features	100.00%
With-PCA	<code>PCA(n_components=0.95)</code> (10 components, typical)	95.21%

7 Machine Learning Algorithms

Ten classifiers were implemented, each wrapped in a `scikit-learn` pipeline (`StandardScaler` → `[PCA]` → `Model`):

- **SVM:** Finds a maximum-margin hyperplane; linear and RBF kernels were tuned.
- **Naive Bayes (Gaussian):** A probabilistic classifier assuming feature independence given the class and Gaussian-distributed features.
- **KNN:** Classifies a sample by majority vote among its k nearest neighbours.
- **Logistic Regression:** A linear model that estimates class probability using the sigmoid function.
- **Decision Tree:** Recursively splits the feature space using impurity measures (Gini/entropy).
- **Random Forest:** A bagging ensemble of decision trees trained on bootstrap samples with random feature subsets.
- **AdaBoost:** A boosting ensemble that sequentially reweights misclassified samples.
- **Gradient Boosting:** A boosting ensemble that sequentially fits new trees to the residual errors of previous trees.
- **XGBoost:** A regularized, efficient implementation of gradient boosting.
- **Stacking:** Combines SVM, Naive Bayes and Decision Tree as base learners, with Logistic Regression as the meta-learner.

8 Experimental Setup

Python Version	3.12
Scikit-Learn Version	1.8.0
Other Libraries	XGBoost, NumPy, Pandas, Matplotlib
IDE	Jupyter Notebook
Operating System	Linux (Ubuntu)

9 Hyperparameter Selection

Each model was tuned with `GridSearchCV` using `StratifiedKFold` (5 splits, shuffled, fixed `random_state`), scored on accuracy, for both the No-PCA and With-PCA settings. Table 3 lists the best parameters found for each model and setting.

Table 3: Best hyperparameters selected by `GridSearchCV`.

Model	Setting	Best Parameters
SVM	No-PCA	<code>C=10</code> , <code>gamma=scale</code> , <code>kernel=rbf</code>
SVM	With-PCA	<code>C=0.1</code> , <code>kernel=linear</code>
Naive Bayes	No-PCA	<code>var_smoothing=1e-09</code>

Model		Setting	Best Parameters	
Naive Bayes		With-PCA	var_smoothing=1e-09	
KNN		No-PCA	metric=manhattan, weights=uniform	n_neighbors=3,
KNN		With-PCA	metric=euclidean, weights=uniform	n_neighbors=3,
Logistic	Regression	No-PCA	C=1	
Logistic	Regression	With-PCA	C=1	
Decision Tree		No-PCA	criterion=entropy, max_depth=3	
Decision Tree		With-PCA	criterion=entropy, max_depth=5	
Random Forest		No-PCA	max_depth=5, n_estimators=100	max_features=sqrt,
Random Forest		With-PCA	max_depth=10, n_estimators=200	max_features=sqrt,
AdaBoost		No-PCA	learning_rate=0.5, n_estimators=100	
AdaBoost		With-PCA	learning_rate=1.0, n_estimators=100	
Gradient	Boosting	No-PCA	learning_rate=0.1, n_estimators=200	max_depth=2,
Gradient	Boosting	With-PCA	learning_rate=0.05, n_estimators=200	max_depth=2,
XGBoost		No-PCA	learning_rate=0.05, n_estimators=200	max_depth=3,
XGBoost		With-PCA	learning_rate=0.1, n_estimators=200	max_depth=5,
Stacking		No-PCA	final_estimator__C=10	
Stacking		With-PCA	final_estimator__C=10	

10 5-Fold Cross-Validation Results

Table 4 and Table 5 report the individual fold accuracies (from the best hyperparameter combination found by GridSearchCV), together with the average and standard deviation across the 5 folds, for the No-PCA and With-PCA settings respectively.

Table 4: 5-fold CV accuracy – No-PCA setting.

Model	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Average	Std. Dev.
SVM	0.967	0.989	0.967	0.978	0.978	0.976	0.008
Naive Bayes	0.956	0.978	0.912	0.912	0.934	0.938	0.026
KNN	0.967	1.000	0.967	0.956	0.989	0.976	0.016
Logistic Regression	0.956	0.967	1.000	0.978	0.967	0.974	0.015
Decision Tree	0.934	0.956	0.901	0.901	0.934	0.925	0.021
Random Forest	0.967	1.000	0.945	0.945	0.978	0.967	0.021
AdaBoost	0.956	0.967	0.967	0.978	0.989	0.971	0.011
Gradient Boosting	0.967	0.978	0.967	0.967	0.989	0.974	0.009
XGBoost	0.967	0.978	0.956	0.967	0.989	0.971	0.011
Stacking	0.967	0.978	0.989	0.956	0.978	0.974	0.011

Table 5: 5-fold CV accuracy – With-PCA setting.

Model	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Average	Std. Dev.
SVM	0.967	0.967	0.967	0.978	0.989	0.974	0.009
Naive Bayes	0.912	0.978	0.890	0.879	0.923	0.916	0.034
KNN	0.978	0.978	0.956	0.956	0.989	0.971	0.013
Logistic Regression	0.956	0.956	1.000	0.978	0.989	0.976	0.018
Decision Tree	0.901	0.934	0.901	0.923	0.967	0.925	0.024
Random Forest	0.967	0.978	0.945	0.934	0.956	0.956	0.016
AdaBoost	0.945	0.989	0.978	0.956	0.989	0.971	0.018
Gradient Boosting	0.945	0.978	0.923	0.956	0.967	0.954	0.019
XGBoost	0.967	0.978	0.967	0.978	0.978	0.974	0.005
Stacking	0.956	0.978	0.989	0.956	0.978	0.971	0.013

11 No-PCA vs With-PCA Comparison

Table 6 compares the mean $\pm$ standard deviation of the 5-fold CV accuracy for every model in both settings, and the change in CV accuracy when PCA is applied.

Table 6: No-PCA vs With-PCA – 5-fold CV accuracy comparison.

Model	CV Accuracy (No-PCA)	CV Accuracy (With-PCA)	Change
SVM	0.9758 ± 0.0082	0.9736 ± 0.0088	-0.0022
Naive Bayes	0.9385 ± 0.0256	0.9165 ± 0.0345	-0.0220
KNN	0.9758 ± 0.0162	0.9714 ± 0.0132	-0.0044
Logistic Regression	0.9736 ± 0.0149	0.9758 ± 0.0176	+0.0022
Decision Tree	0.9253 ± 0.0213	0.9253 ± 0.0245	+0.0000
Random Forest	0.9670 ± 0.0208	0.9560 ± 0.0155	-0.0110
AdaBoost	0.9714 ± 0.0112	0.9714 ± 0.0178	+0.0000
Gradient Boosting	0.9736 ± 0.0088	0.9538 ± 0.0189	-0.0198
XGBoost	0.9714 ± 0.0112	0.9736 ± 0.0054	+0.0022
Stacking	0.9736 ± 0.0112	0.9714 ± 0.0132	-0.0022

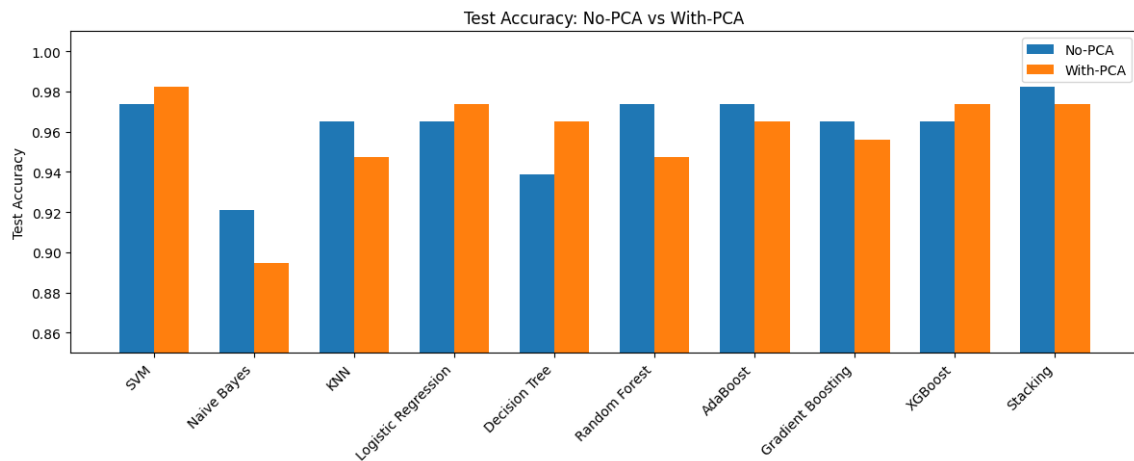


Figure 3: Test accuracy comparison: No-PCA vs With-PCA, for all 10 models.

Inference: PCA does not uniformly help or hurt every model. Decision Tree, Logistic Regression, SVM and XGBoost show a small improvement in test accuracy with PCA, while Naive Bayes, KNN, Random Forest, AdaBoost, Gradient Boosting and Stacking perform the same or slightly worse with PCA. This is examined further in the Observation Questions and Discussion sections.

12 Test Set Performance

Table 7 reports accuracy, precision, recall, F1-score and ROC-AUC on the held-out 20% test set (114 samples) for every model in both settings.

Table 7: Test-set performance for all 10 models, No-PCA vs With-PCA.

Model	Setting	Accuracy	Precision	Recall	F1	ROC-AUC
SVM	No-PCA	0.9737	1.0000	0.9286	0.9630	0.9927
SVM	With-PCA	0.9825	1.0000	0.9524	0.9756	0.9967

Model	Setting	Accuracy	Precision	Recall	F1	ROC-AUC
Naive Bayes	No-PCA	0.9211	0.9231	0.8571	0.8889	0.9891
Naive Bayes	With-PCA	0.8947	0.8571	0.8571	0.8571	0.9613
KNN	No-PCA	0.9649	1.0000	0.9048	0.9500	0.9735
KNN	With-PCA	0.9474	0.9737	0.8810	0.9250	0.9780
Logistic Regression	No-PCA	0.9649	0.9750	0.9286	0.9512	0.9960
Logistic Regression	With-PCA	0.9737	1.0000	0.9286	0.9630	0.9970
Decision Tree	No-PCA	0.9386	0.9268	0.9048	0.9157	0.9883
Decision Tree	With-PCA	0.9649	1.0000	0.9048	0.9500	0.9628
Random Forest	No-PCA	0.9737	1.0000	0.9286	0.9630	0.9950
Random Forest	With-PCA	0.9474	0.9500	0.9048	0.9268	0.9907
AdaBoost	No-PCA	0.9737	1.0000	0.9286	0.9630	0.9858
AdaBoost	With-PCA	0.9649	0.9750	0.9286	0.9512	0.9934
Gradient Boosting	No-PCA	0.9649	1.0000	0.9048	0.9500	0.9934
Gradient Boosting	With-PCA	0.9561	0.9744	0.9048	0.9383	0.9911
XGBoost	No-PCA	0.9649	1.0000	0.9048	0.9500	0.9947
XGBoost	With-PCA	0.9737	0.9756	0.9524	0.9639	0.9931
Stacking	No-PCA	0.9825	1.0000	0.9524	0.9756	0.9987
Stacking	With-PCA	0.9737	1.0000	0.9286	0.9630	0.9957

The best test accuracy (98.25%) is achieved jointly by **SVM (With-PCA)** and **Stacking (No-PCA)**. Stacking (No-PCA) also has the highest ROC-AUC of all 20 model/setting combinations (0.9987).

13 Result Visualization

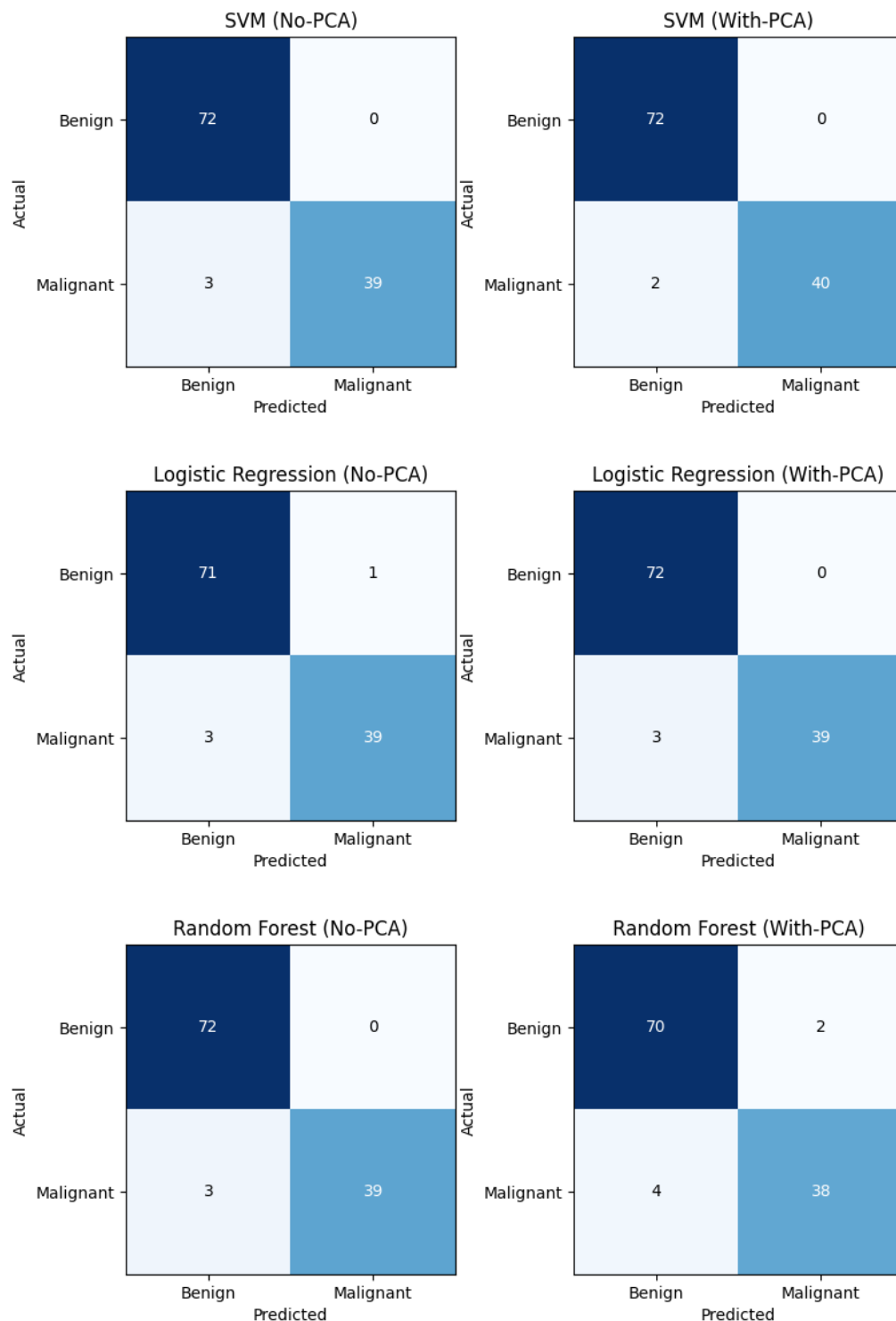


Figure 4: Confusion matrices for SVM, Logistic Regression and Random Forest, No-PCA vs With-PCA.

Inference: SVM and Logistic Regression make slightly fewer errors With-PCA (fewer malignant cases misclassified as benign), while Random Forest makes slightly more errors With-PCA. This matches the accuracy differences reported in Table 7.

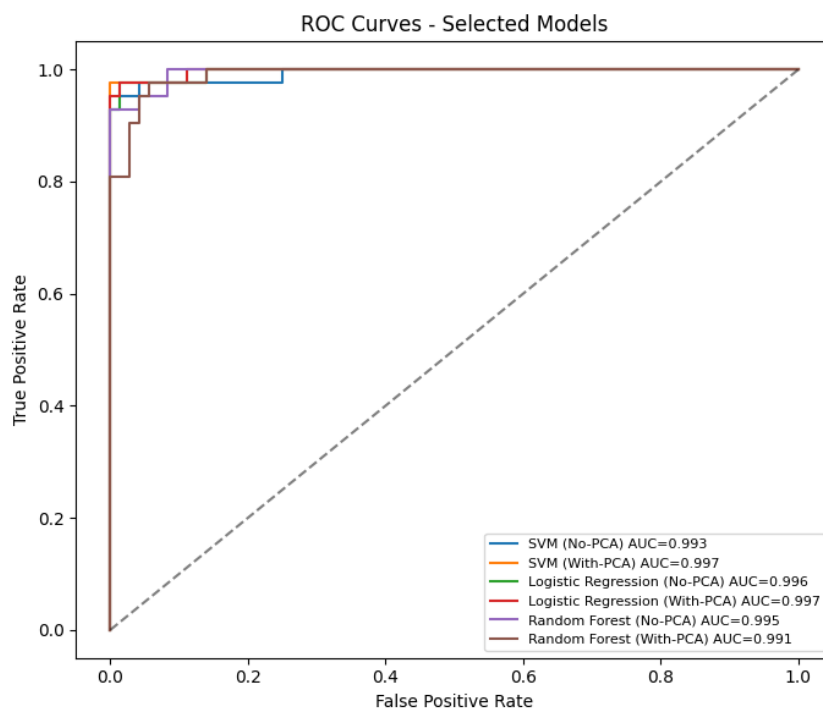


Figure 5: ROC curves for SVM, Logistic Regression and Random Forest, No-PCA vs With-PCA.

Inference: All six curves stay close to the top-left corner, confirming that every one of these models separates the two classes very well (ROC-AUC above 0.99 in all six cases). The curves for the No-PCA and With-PCA versions of each model are close to each other, showing PCA does not meaningfully change ranking quality for these particular models.

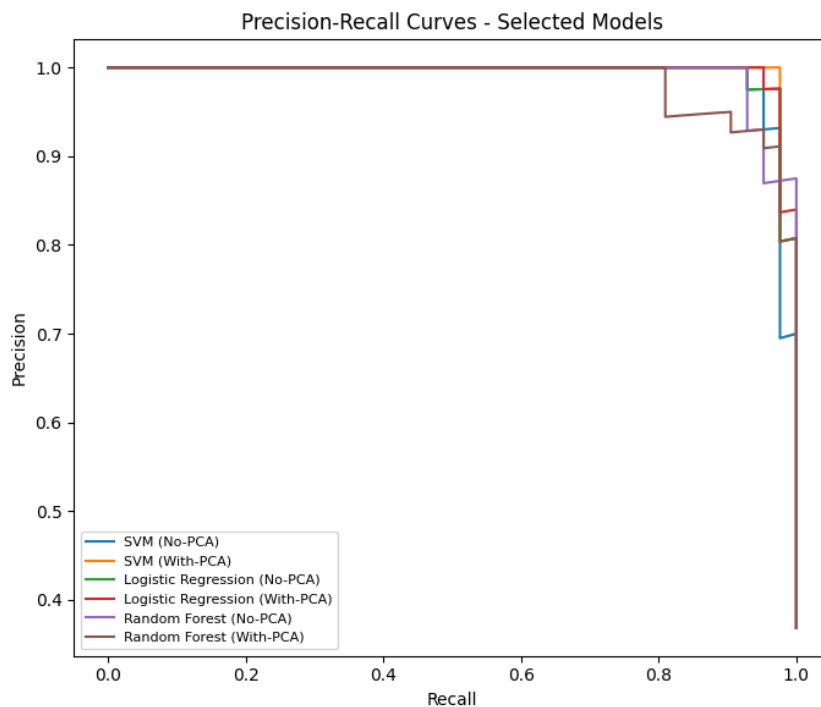


Figure 6: Precision-Recall curves for SVM, Logistic Regression and Random Forest, No-PCA vs With-PCA.

Inference: Precision stays high (close to 1.0) over most of the recall range for all six curves, which is expected given the strong ROC-AUC scores and the moderate class imbalance (62.7% vs 37.3%).

14 Stability Analysis

Model stability is measured using the standard deviation of the 5-fold CV accuracy: a lower standard deviation means more consistent performance across folds.

Table 8: Stability comparison (5-fold CV standard deviation).

Model	Std. Dev. (No-PCA)	Std. Dev. (With-PCA)	More Stable Setting
SVM	0.0082	0.0088	No-PCA
Naive Bayes	0.0256	0.0345	No-PCA
KNN	0.0162	0.0132	With-PCA
Logistic Regression	0.0149	0.0176	No-PCA
Decision Tree	0.0213	0.0245	No-PCA
Random Forest	0.0208	0.0155	With-PCA
AdaBoost	0.0112	0.0178	No-PCA
Gradient Boosting	0.0088	0.0189	No-PCA
XGBoost	0.0112	0.0054	With-PCA
Stacking	0.0112	0.0132	No-PCA

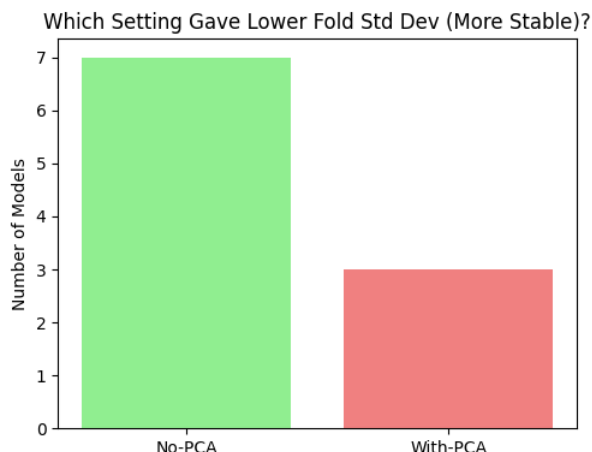


Figure 7: Number of models that were more stable (lower fold std. dev.) in each setting.

Inference: Only 3 of the 10 models (KNN, Random Forest, XGBoost) were more stable With-PCA; the other 7 models had a lower fold-to-fold standard deviation without PCA. So PCA did not systematically improve cross-validation stability on this dataset.

15 Observation Questions

1. Which models improved most with PCA? Which did not? Why?

Decision Tree improved the most (test accuracy 93.86% $\rightarrow$ 96.49%, +2.63%). Logistic Regression, SVM and XGBoost each improved by +0.88%. Naive Bayes and Random Forest were hurt the most (both -2.63%), and KNN dropped by -1.75%. Decision Tree, SVM and Logistic Regression benefit because PCA removes correlated/redundant raw features and gives cleaner, decorrelated decision boundaries. Naive Bayes assumes conditional feature independence given the class; PCA components are linear mixtures of the original features, which can break this assumption and hurt performance. Random Forest already copes well with high-dimensional, redundant features through random feature subsampling, so compressing 30 features into 10 components removes some of the redundant signal it was using.

2. Did PCA reduce variance across folds and produce more stable results?

No, not in general. Only 3 of the 10 models (KNN, Random Forest, XGBoost) had a lower CV standard deviation With-PCA (Table 8); for the remaining 7 models the fold standard deviation was slightly higher With-PCA. PCA's effect on stability is model-dependent rather than a consistent improvement here.

3. For high-dimensional data, was PCA beneficial in reducing overfitting?

Only partially. Comparing the gap between CV accuracy and test accuracy, the gap shrank for 5 of the 10 models after PCA and grew for the other 5. WDBC has 30 features and 569 samples (about 19 samples per feature), which is only moderately high-dimensional, so the models were not overfitting severely to begin with – this limited how much PCA could help with overfitting.

4. How did linear models such as Logistic Regression and SVM behave compared to ensemble models with PCA?

Both linear/margin-based models improved slightly with PCA (SVM: 97.37% $\rightarrow$ 98.25%; Logistic Regression: 96.49% $\rightarrow$ 97.37%), consistent with the idea that removing multicollinearity among correlated features helps a linear decision boundary. The ensemble

models mostly performed the same or worse with PCA (Random Forest -2.63%, Gradient Boosting -0.88%, AdaBoost -0.88%), since tree-based ensembles already model non-linear feature interactions and select useful features internally, so compressing the feature space can discard information they were using effectively.

5. Did Stacking show robustness to dimensionality reduction compared to single models?

Stacking's test accuracy dropped only slightly with PCA (98.25% $\rightarrow$ 97.37%, -0.88%), a much smaller drop than Naive Bayes (-2.63%), Random Forest (-2.63%) or KNN (-1.75%). Stacking (No-PCA) achieved the joint-best test accuracy (98.25%, tied with SVM With-PCA) and the best ROC-AUC of all 20 model/setting combinations (0.9987), showing it is a consistently strong performer, though – like most ensembles here – it did not actually benefit from PCA the way the linear models did.

16 Discussion

PCA compressed the 30 WDBC features into 10 principal components while retaining 95.21% of the variance, but its effect on model performance was mixed rather than uniformly positive. Simple, linear or single-tree models (Logistic Regression, SVM, Decision Tree) tended to benefit slightly from the decorrelated, lower-dimensional input, while models that already handle correlated or high-dimensional data well internally (Random Forest, boosting methods, Stacking) tended to lose a small amount of accuracy after PCA. Naive Bayes was the most negatively affected, likely because PCA components violate its conditional-independence assumption more than the original (already fairly separable) raw features do.

Cross-validation stability, measured by the standard deviation across folds, was not systematically improved by PCA either – only 3 of 10 models became more stable. Because WDBC is only moderately high-dimensional relative to its sample size, the main motivation for PCA here (dimensionality reduction and noise removal) did not translate into a large, consistent gain for most models; the dataset's 30 features were already informative and only mildly redundant.

Overall, the best-performing configurations were SVM With-PCA and Stacking No-PCA (both 98.25% test accuracy), with Stacking No-PCA reaching the highest ROC-AUC (0.9987) recorded in the experiment.

17 Conclusion

PCA reduced the WDBC feature space from 30 features to 10 principal components (95.21% variance retained), applied inside the pipeline with a 95% variance target so it is refit independently in every cross-validation fold. PCA did not help uniformly: it improved Decision Tree (the most), Logistic Regression, SVM and XGBoost, but slightly hurt Naive Bayes, KNN, Random Forest, AdaBoost, Gradient Boosting and Stacking. It also did not make cross-validation results consistently more stable, improving fold-to-fold stability for only 3 of the 10 models. The best overall test accuracy (98.25%) was achieved jointly by SVM (With-PCA) and Stacking (No-PCA), and Stacking (No-PCA) gave the best ROC-AUC (0.9987). Based on these results, dimensionality reduction with PCA should not be applied by default – its benefit depends on the model: it tends to help linear/margin-based models and single decision trees on this dataset, while ensemble

methods that already handle redundant, high-dimensional features well may lose useful information when the feature space is compressed.

18 References

<https://github.com/sugith-2025/MachineLearning/tree/main/Assignment-7>.